

FOODBRIDGE — DAY 9 NOTES

React Frontend Mentorship

Date: [Today's Date] | Topic: Axios Interceptor, Auto-Logout & Login Redirect

Today we wired up the Axios interceptor to automatically attach the JWT to every request, added auto-logout when the backend returns 401, and fixed the login redirect with { replace: true }.

1. WHAT IS AN INTERCEPTOR?

An interceptor is a function that runs automatically **before every request** or **after every response**.

Think of it as a security checkpoint:

- Before request leaves your app → check if token exists → add it
- After response comes back → check if token is expired → logout



2. REQUEST INTERCEPTOR (BEFORE REQUEST)

This runs **BEFORE** every request is sent.

```
axiosInstance.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem("token");
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);
```

Line-by-line explanation:

Line	What it does
axiosInstance.interceptors.request.use(	Intercept every outgoing request
(config) => {	config = the request object (url, headers, body...)
localStorage.getItem("token")	Check for a saved JWT token
if (token) {	Only add header if token actually exists
config.headers.Authorization = `Bearer \${token}`	Add token as Authorization header
return config;	Send the (now modified) request

Why we need this:

Endpoint	Needs Token?
POST /users (Register)	■ No (public)
POST /users/login (Login)	■ No (public)
POST /donations (Create)	■ Yes (only DONOR)
GET /donations/available	■ Yes (only NGO)
PUT /donations/{id}/accept	■ Yes (only NGO)

- **Without interceptor:** manually add token to EVERY request
- **With interceptor:** token is added automatically

3. RESPONSE INTERCEPTOR (AFTER RESPONSE)

This runs **AFTER** every response.

```
axiosInstance.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem("token");
      toast.error("Session expired. Please login again.");
      window.location.href = "/login";
    }
    return Promise.reject(error);
  }
);
```

Line-by-line explanation:

Line	What it does
(response) => response	If success → pass response through unchanged
(error) => {	If any error occurs...
error.response?.status === 401	Check if the error is 401 Unauthorized
localStorage.removeItem("token")	Clear the expired token
toast.error("Session expired...")	Show error notification to user
window.location.href = "/login"	Hard-redirect to login page

How auto-logout works step by step:

- User logs in → token saved (expires after set time)
- Token expires
- User tries to access a protected endpoint
- Backend returns **401 Unauthorized**
- Response interceptor catches the 401
- Clears token → shows toast → redirects to /login

4. WHAT 401 VS 403 MEANS

Status	Meaning	What it tells you
401 Unauthorized	"I don't know who you are"	No token or invalid/expired token
403 Forbidden	"I know you, but you can't do this"	Valid token but wrong role / no permission

In testing:

- Getting **403** = Token was sent and validated ■
- Getting **401** = Token not sent or invalid ■

5. NAVIGATE WITH { REPLACE: TRUE }

What it does:

- Replaces current page in browser history instead of adding a new entry
- Prevents back button from going back to the login page

Scenario	History stack behaviour
Without replace	Home → Login → Dashboard → Back → Login ■
With replace	Home → [Login replaced by Dashboard] → Back → Home ■

```
// Without replace
navigate(`/ ${decoded.role.toLowerCase()}/dashboard`);
```

```
// With replace
navigate(`/ ${decoded.role.toLowerCase()}/dashboard`, { replace: true });
```

When to use replace:

Situation	Use replace?
After login	■ Yes
After logout	■ Yes
After form submission	■ Yes
Regular link navigation	■ No

6. FULL AXIOSINSTANCE.JS

```
import axios from "axios";
import { toast } from "react-toastify";

const axiosInstance = axios.create({
  baseURL: "http://localhost:8080",
});

// Request Interceptor - adds token to every request
axiosInstance.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem("token");
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
  },
  (error) => Promise.reject(error)
);
```

```
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// Response Interceptor - handles token expiry
axiosInstance.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem("token");
      toast.error("Session expired. Please login again.");
      window.location.href = "/login";
    }
    return Promise.reject(error);
  }
);

export default axiosInstance;
```

7. TESTING THE INTERCEPTOR

How we tested:

- Added test code to Donor Dashboard
- Made a request to a protected endpoint
- Checked Network tab in DevTools for the Authorization header

What we saw:

- Authorization: Bearer header present in every request
- 403 Forbidden response (correct – DONOR cannot access ADMIN endpoint)
- Conclusion: Interceptor is WORKING

8. INTERVIEW-READY ANSWERS

Q: What is an Axios interceptor and why would you use it?

A: An Axios interceptor is a function that intercepts requests before they are sent or responses before they are handled. In our project, we use a request interceptor to automatically add JWT tokens to every API request, saving us from manually attaching the Authorization header each time. We also use a response interceptor to handle token expiry — when the backend returns a 401 status, we automatically clear the token and redirect the user to the login page.

Q: What's the difference between 401 and 403 status codes?

A: 401 Unauthorized means the client has not provided valid credentials — either no token was sent or the token is invalid/expired. 403 Forbidden means the client is authenticated but does not have permission to access the resource. In our project, getting a 403 confirms that the JWT was sent and validated — the interceptor is working correctly; the issue is just role-based access.

Q: Why do you use { replace: true } in navigate()?

A: Using { replace: true } replaces the current entry in the browser's history stack instead of pushing a new one. We use this after login so that if the user presses the back button after reaching their dashboard, they go to the page before login (like Home) rather than being taken back to the login form — which would be a confusing UX.

9. WHAT WE COMPLETED TODAY

- Request interceptor – auto-adds JWT to every request
- Response interceptor – auto-logout on 401
- Login redirect with { replace: true }
- Tested interceptor with real endpoints

10. GIT COMMAND

```
git commit -m "feat: add axios interceptor for JWT and auto-logout on 401"
```

11. NEXT STEPS (DAY 10)

Start building Donor Dashboard:

- Create Donation form with image upload
- Fields: food name, quantity, expiry date, description

- POST /donations with multipart/form-data

END OF DAY 9 NOTES